导览机器人系统 — 面试 QA 总表（口语背诵版）

这份怎么用

- 答案都是**能直接念出口的话**，一段 3~5 句，念完不超过 30 秒。
- 面试官**看不到你的代码**，他问的是「**你怎么设计的、为什么、踩过什么坑、换你会怎么改**」。文件名、话题名、参数名一律不出口，都挪进「📌 自己复习」。
- **题的顺序就是面试顺序**：先问是什么 → 再问为什么 → 再抠细节 → 再问底层 → 最后问怎么改。
- **追问都是从上一句答案里长出来的**：你答案里提到什么技术，他下一句就问那个技术是什么。答的时候心里要留好钩子。
- 答不上来的题，**主动划边界 + 给改进方案**，比硬编强。

覆盖范围：简历「基于 ROS 2 与 Nav2 的智能导览机器人系统」全部六条——① 节点架构 ② 定位与导航 ③ 状态机调度 ④ 建模与仿真 ⑤ 访客检测 ⑥ 语音播报与问答。

🔴 **六条红线（全文通用，背题前先看）**

1. 开口第一句就说「**这是仿真项目**」，**没上真机**。别等被追问出来。
2. **导航算法不是我写的**：Nav2 是开源导航栈，我做的是**集成 + 调参 + 跑通**。被问原理就答「知道它干什么，没自己实现」。
3. **迎宾默认是距离触发**，不是摄像头识别人。视觉分支默认关、没验证过。
4. **问答是关键词规则匹配**，不是大模型、不是语义理解。语音输入是**模拟**的，没有真实 ASR。
5. **地图是脚本预置生成的**，没跑 SLAM 建图。别把「已知地图定位」说成「自主建图导航」。
6. **导航失败是「跳过该展点」**，不是回退重试、不是状态回滚。

目录

- 第 0 部分：开场自述（3 题）
- 条目一：ROS 2 节点架构（16 题）
- 条目二：定位与导航 Nav2（22 题）
- 条目三：导航与导览调度（19 题）
- 条目四：建模与仿真（20 题）
- 条目五：访客检测与迎宾（11 题）
- 条目六：语音播报与问答（12 题）
- 附录：通用追问（6 题）
- 附录二：简历名词八股（26 题）
  - ROS2 通信底层（8 题）
  - C++ 语言八股（8 题）
  - Python 语言八股（4 题）
  - 网络与并发八股（6 题）

第 0 部分：开场自述（3 题）

Q0.1 简单介绍一下你这个项目是干什么的？ [必背]

答： 这是一个面向展厅讲解的**导览机器人**，跑在 ROS 2 上。机器人在展厅里依次走到四个展区，到站播讲解词、回答问题，走完一轮回到起点。业务层是 Python 写的，跑在 Ubuntu 虚拟机里，用 Gazebo 做仿真。 我主要做了四块：**机器人模型和展厅场景的搭建**、**Nav2 导航栈的集成和调参**、**导览状态机的调度**，还有**迎宾检测、语音播报、问答**这几个业务节点。

追问：这是真机吗？

> **不是，是仿真，没有上真机。** 我主动先说清楚这点——整个流程是在 Gazebo 里跑通的。

追问 2：为什么不上真机？

> 个人条件下没有实体机器人。仿真也有仿真的价值：模型、传感器、场景都可控，可以反复复现问题，先把**上层业务逻辑**跑通——导航怎么调、状态机怎么切、节点之间怎么通信。真机要补的是底盘驱动、传感器标定和时间同步，那是另一层的事。

Q0.2 你说的「导航」和「导览」是一回事吗？ [必背] ★ 高频

答： **不是，是两层。** 导航是 Nav2 干的，回答「怎么从 A 点走到 B 点」；导览是我写的状态机，回答「下一站去哪、到了说什么、什么时候继续」。 两层之间靠一个 **Action** 衔接：我把目标点的坐标发下去，Nav2 走完把结果回调给我。这么拆的好处是：换导航算法不用动业务，加一个展点也不用动导航。

追问：那导航这部分是你写的吗？

> **算法不是我写的。** Nav2 是开源成熟导航栈，我做的是**集成和调参**——选哪个规划器、代价地图怎么配、避障留多宽、速度给多少，这些是我对着自己的场景调出来的。**原理层面我知道每个模块干什么，但没有自己实现算法。**

Q0.3 这个项目你一个人做的吗？哪些是别人的？ [必背] ★★ 开场必被问

答： 一个人做的，但**是站在别人的东西上做的**。底盘模型是拿开源项目改的，Gazebo 是官方仿真器，Nav2 是官方导航栈。 我自己的部分是：**展厅场景和地图的生成脚本、导览状态机、五个业务节点的拆分和通信、Nav2 的参数配置，还有启动脚本和可视化工具。** 一句话：**算法和工具用现成的，业务集成和场景搭建是我做的。**

> 💡 这题的打分点在**边界清晰**。说「我用了 X、自己写了 Y」，比含糊说「这个项目我都做过」靠谱一百倍。

📌 自己复习：底盘来源 `akhiljithvg/Autonomous-Navigation-Robot`；原项目的 RK3588、串口、CAN 那些能力不属于本项目，别拿来充数。

条目一：ROS 2 节点架构

> **简历原话** 按业务拆分导览状态机、语音播报、问答、访客检测、语音输入 5 个节点，通过 Topic、Service、Action 完成状态传递、业务查询与导航目标下发；使用 Launch、YAML 管理节点启动、话题映射与展点配置。
>
> **本条三个技术点**：① 为什么拆成多个节点（5 题） ② Topic / Action / Service 三种通信（6 题） ③ Launch 与 YAML 配置驱动（5 题）

技术点 1：为什么拆成多个节点（5 题）

Q1-1 为什么要把一个程序拆成好几个节点？写成一个不行吗？ [必背] ★ 高频

答： 写成一个也能跑，但**塌不了的好处全没了**。我们是五个独立进程：**主控状态机、语音播报、问答、迎宾检测、语音输入**。 拆开有三点好处：一是**隔离故障**，播报那个节点崩了不影响导航继续跑；二是**能单独换**，我想把离线语音换成别的引擎，只换那一个节点；三是**能单独看**，哪个环节出问题直接看那个节点的日志。 代价是多进程通信有开销，但这个场景数据量极小，完全不是瓶颈。

追问：那你是按什么维度拆的？

> **按业务职责拆，一个节点干一件事。** 主控只管调度，播报只管发声，问答只管匹配规则。这样判断起来很简单：**一件事需要独立重启、独立调试、或者将来可能单独换掉，就值得独立成节点。**

Q1-2 五个节点之间是完全独立的吗？有没有依赖关系？ [加分]

答： **运行上独立，业务上有顺序要求。** 它们不共用内存、不互相调用，靠话题收发消息，所以任何一个单独启动都不会报错。 但业务上有先后：**主控要先等到导航服务就绪才开工**，迎宾节点只在「等待访客」这个状态下才检测，问答节点收到问题才回答。这个顺序是靠**主控广播出来的状态**来协同的，不是靠启动顺序硬保证的。所以启动顺序反了也不会崩，只是会一直等在那儿。

Q1-3 节点之间会不会互相阻塞？比如播报很慢会不会拖住导航？ [加分] ★★

答： **不会，因为用的是异步发布订阅。** 主控把讲解词往话题上一发就立刻返回，根本不等播报完。 这一点其实是双刃剑：好处是**不阻塞**，坏处是**没有完成确认**——主控不知道这句话到底念完了没有。我现在是靠**配一个固定停留时间**来近似，讲解完等几秒再进下一站。 这是我认知到的一个不足：严格来说应该让播报节点**发一个"念完了"的事件**回来，主控收到才继续。现在的做法在讲解词长短不一时会不准。

Q1-4 如果某个节点挂了，系统会怎样？ [加分] ★ 有实战感

答： **会卡在等那一步，不会崩，但也不会自愈。** 比如播报节点挂了，主控还在正常发讲解词，只是没人念；问答节点挂了，提问就永远等不到答案，靠**超时兜底**进下一站。 所以我在设计上给了两道保险：一是**问答有超**

时，超时就自己往下走；二是**每个环节都有日志**，卡在哪一步一眼能看出来。 但坦白说没有做**看门狗**——真要产品化，应该监控每个节点的心跳，挂了自动拉起。这是我觉得最该补的一点。

Q1-5 这个节点划分有什么可以改进的? [加分] ★ 收尾必问

答： 两点。第一，**缺节点健康监控**：现在节点挂了只能靠人看日志，应该有心跳和自动拉起。 第二，**主控承担了太多**：它既管状态机，又负责发讲解词、还要处理人工指令。理想情况应该把「人工接管」那部分拆成一个独立的指令节点，主控只管自动流程。 核心想法是：**节点拆分的粒度，应该配合上可观测性**，光拆开但没人知道它活着没有，等于只做了一半。

技术点 2：Topic / Action / Service 三种通信（6 题）

Q2-1 你项目里 Topic、Action、Service 分别用在哪儿? [必背] ★★ 高频

答： **Topic 做状态广播**——主控把当前状态、讲解词、当前展点发到话题上，谁关心谁订阅，异步、不等结果。 **Action 用在导航上**——因为导航是**长任务**，要持续几十秒，还得知道「接没接受、走到哪了、成功还是失败」，这正好是 Action 的能力。 **Service 主要出现在 Nav2 生命周期节点的管理上**——启动脚本要调服务去查询和切换那几个导航节点的状态。我自己的业务节点之间**没有直接的同步调用需求**，所以没写。

> ❗ **别急着把三者都说成"我大量使用"**。准确说法：业务逻辑是 Topic + Action，Service 是导航栈生命周期那层在用。

Q2-2 为什么导航要用 Action? 用 Topic 发个目标点不行吗? [必背] ★★ 核心八股

答： **不行，因为 Topic 是单向的、没有结果。** 我只发一个目标点出去，就永远不知道它到没到、是成功了还是卡住了。 Action 本质上是**两个话题加一个服务**：一个发目标、一个收反馈、一个取结果，还带**取消**能力。导航刚好需要这三样——要知道有没有被接受、要能中途叫停、要拿最终结果才能决定下一步。 一句话：**Topic 是「喊一嗓子」，Service 是「一问一答」，Action 是「派个活、随时汇报、最后交结果」。**

追问：那被接受就等于成功了吗?

> **不是，这是最经典的坑。** 目标被接受只说明服务端收下了，**不代表能走到**。我是等**最终结果回调**回来，判断状态是成功才进讲解。

Q2-3 那 Action 的反馈你怎么用的? [加分]

答： 说实话我**只用了结果，没用中间反馈**。Action 支持周期性回报进度（比如离目标还有多远），我在代码里留了这个回调，但调试阶段没实际往里填东西。 原因是：**业务上我只需要"到了没有"这个二值结果**，中间进度对状态机没用。但如果要做界面上的进度条、或者做"走了一半发现堵住就换路线"，那就得用反馈了。 我知道这个能力存在，只是当前没用到——**知道能力边界在哪，和用过一样重要。**

Q2-4 发布订阅模型有什么缺点？ [加分]

答： 三个。第一，**发布方不知道有没有人收**：我发一条讲解词出去，没人订阅也不报错，出了问题不好查。 第二，**没有完成确认**：前面说的播报，我发完就走，不知道对面做完没有。 第三，**消息可能丢**：订阅方如果处理慢或者刚启动，早发的消息可能就错过了。我给队列长度设了个小缓冲，但严格讲关键状态应该配**持久化策略**，让后启动的节点也能拿到最后一条。 这也是为什么「状态」这种关键信息我会**周期广播**而不是只发一次。

Q2-5 同一个话题有多个订阅者，数据会被复制吗？ [加分]

答： **数据不会复制，是各自反序列化出自己的一份。** 发布方序列化成字节发出去，每个订阅方收到字节后在自己进程里还原成对象，所以**两边操作的是不同内存里的两个对象**，互不影响。 好处是解耦彻底：加一个订阅者，发布方一行代码都不用改。代价是**多份内存和多次序列化开销**——传图像这种大数据时这个开销很可观，所以大消息一般用共享内存或者零拷贝传输。 我们传的是字符串和坐标，数据量极小，完全不用考虑这个。

Q2-6 节点之间传消息，线程安全谁来保证？ [加分]

答： **通信中间件保证收发本身是安全的，但业务代码里的共享数据要自己保护。** 我这个项目里每个节点基本是**单线程 + 回调**的模型：一个定时器在跑主循环，订阅回调也是在同一个执行器上被串行调用的。所以我在回调里只做一件事——**把数据存下来**，真正处理放到主循环里做。 好处是**避免了回调和主循环并发改同一份数据**，不需要加锁。这是个很实用的习惯：**回调只做记录，处理留给主循环。**

技术点 3：Launch 与 YAML 配置驱动（5 题）

Q3-1 为什么要写 Launch 文件？一个个手动启动不行吗？ [必背]

答： **不行，节点太多了。** 这一套跑起来要起十几个进程：仿真、桥接、定位、规划、控制、生命周期管理，再加我的五个业务节点。手动起要开一堆终端，还得保证顺序。 Launch 的好处是：**一条命令起全部，还能统一传参数。**我可以把所有节点、它们的参数、启动顺序写在一个 Python 文件里，一条命令拉起来。 另外它还支持**声明启动参数**，比如我可以在命令行决定用仿真迎宾还是摄像头迎宾，不用改代码。

Q3-2 你说的「YAML 配置驱动」，具体配了哪些东西？ [必背]

答： 两层。**业务层**配的是展点和问答规则：**五个展点的坐标、朝向、讲解词、提问提示词**，还有**关键词到答案的匹配规则表。 算法层**配的是 Nav2 那一大票参数：定位的粒子数、规划器选型、代价地图的膨胀半径、速度上限、采样数，还有仿真和 ROS 之间的话题桥接映射。 好处是：**加一个展点、改一句讲解词、加一条问答规则，全都是改 YAML，业务代码一行不动。**

追问：那你改完配置要重新编译吗？

> 改 YAML **不用编译，但要重启对应节点。**因为配置是**节点启动时读进去的**，运行中不会自动重新加载。这一点我踩过坑——改完配置发现没生效，就是因为没重启。

Q3-3 展点坐标和朝向是怎么表示朝向的? [加分] ★ 容易答错

答： 坐标是**平面 xy 加一个 yaw 角**，yaw 用**弧度**表示，0 是朝正 x 方向，π 是朝负 x 方向。 代码里把 yaw 转成**四元数**再填进目标位姿——绕竖直轴转的话，只需要填 z 和 w 两个分量，等于 sin 和 cos 各自的半角值。 为什么要转：因为姿态的标准表示是四元数，它避免了欧拉角的万向锁问题，接口只认这个格式。

Q3-4 加一个新展点，你要改哪些地方? [必背] ★ 高频场景题

答： **只改一个配置文件。** 在展点列表里加一项：名字、坐标、朝向、讲解词、提问提示词，**代码一行都不用动。** 如果还要能回答跟这个展点相关的问题，就在问答规则表里**再加一条关键词和答案。** 重启导览节点之后，状态机会自动按列表顺序走到这个新点。这也是我为什么把展点数量做成**读取列表长度**而不是写死——**加一个点就是加一条配置。**

Q3-5 「配置驱动」这种方式有什么代价? [加分]

答： 两点。第一，**错误从编译期挪到了运行期**：坐标填错了、名字写错了，编译完全能过，要跑起来才发现——比如机器人往墙里撞，或者某个展点永远答不上话。 第二，**配置和场景是分离的**：展点坐标配在业务配置里，墙的位置定义在场景脚本里，**两边没有强关联。**我改了墙的位置，坐标不会自动跟着变，得人工确认这个点还在不在空地上。 我的教训是：**凡是有对应关系的多份配置，一定要在改动时一起核对**，最好能加一个启动自检，检查每个目标点是不是落在可通行区域。

条目二：定位与导航 Nav2

> **简历原话** 集成 Nav2 导航栈，配置 AMCL 完成已知地图定位、NavFn（Dijkstra）全局路径规划与 DWB 局部轨迹跟踪；调整全局/局部代价地图障碍层与膨胀层参数实现动态避障，通过 Lifecycle 节点编排导航栈激活顺序。
>
> > ⚠️ **背诵优先级**：真正要背的是「**这一层是集成 + 调参，不是我写算法**」这个定性，以及每个模块**干什么**。具体参数值是储备，说个大概即可。
>
> **本条五个技术点**：① AMCL 定位（5 题） ② 全局规划 NavFn（4 题） ③ 局部规划 DWB（5 题） ④ 代价地图与避障（4 题） ⑤ Lifecycle 编排（4 题）

技术点 1：AMCL 定位（5 题）

Q1-1 机器人怎么知道自己在哪？ [必背]

答： 用的是 **AMCL**，在**已知地图**里做定位。原理上它维护一堆「机器人可能在这儿」的**粒子**：机器人一动，就按运动模型把所有粒子往前挪；再用激光扫描跟地图比对，跟地图吻合的粒子权重升高、不吻合的降低，然后按权重重采样。 跑一会儿之后粒子会收敛到真实位置附近。它解的是「**我知道地图长什么样，我现在在哪**」这个问题。

追问：那没有地图怎么办？

> 那就得先**建图**，也就是 SLAM——一边走一边把地图画出来。我们这个项目**地图是预先生成好的**，没跑 SLAM。配置里有相关文件和启动项，但演示中没运行、也没验证过。

Q1-2 为什么用激光雷达定位，不用摄像头？ [加分]

答： **激光直接给距离，摄像头只给像素。** 定位要的是「墙离我多远」这个几何量，激光一帧就给出几百个方向上的精确距离，几乎不用算；摄像头要先做特征提取、再做深度估计，误差大还挑光照。 另外激光**受光照影响极小**，展厅里明暗变化不影响它；**计算量也小**，对只有 4 核的虚拟机很关键。 代价是激光只是一条水平线，**看不到矮的或者悬空的障碍**。真要产品化，通常会激光加视觉再加超声波做互补。

Q1-3 定位的初始位置是怎么给的？ [加分] ★ 细节加分

答： **配置里直接给了初始位姿**，坐标和朝向都写成参数，启动时加载。 我把它设成了**跟机器人在仿真里的出生位置完全一致**，这样一上来粒子就在正确位置附近，不需要人工在界面上点一下「我在这里」来初始化。 这是个很实用的小技巧：**初始位姿给准，能省掉现场每次手动初始化的麻烦，也避免一开始定位飘掉。**

Q1-4 粒子数为什么设成一个范围而不是固定值? [加分] ★★

答： 这是 AMCL 的一个**自适应机制**：定位还不确定时用**多粒子**去覆盖更大的可能性，收敛之后自动降到**少粒子**省算力。 这个设计很聪明——**粒子数直接决定计算量**，全局搜索的时候舍得花，已经确定位置了就没必要养那么多。我在配置里设的是上限几百、下限一百左右。 对我这种算力紧张的虚拟机来说，这个范围比写死一个值划算得多。

Q1-5 定位会不会飘? 飘了会怎样? [加分]

答： **会。** 典型场景是**长走廊**——前后看上去一模一样，激光特征重复，粒子容易收敛到错误位置；还有**被人群围住**的时候，激光看到的都是腿，跟地图对不上。 飘了的表现是：机器人以为自己在别处，规划出来的路线就是错的，会往墙里撞或者原地打转。 我们的**仿真里几乎不会飘**，因为仿真给的里程计是理想值、没有真实轮子的打滑和编码器漂移。这也是**仿真和真机最大的差距之一**——真机上这块必须靠调参和加传感器融合来扛。

技术点 2：全局规划 NavFn（4 题）

Q2-1 全局规划是干什么的? 用的什么算法? [必背] ★ 高频

答： 全局规划回答「**从当前位置到目标点，大体走哪条路**」。它在**全局代价地图**上做搜索，输出一条完整的路径点序列。 用的是 **NavFn 这个规划器，配的是 Dijkstra**，配置里那个 A* 选项是关掉的。 所以被问到是 A* 还是 Dijkstra，*明确答 Dijkstra，别顺着话头说 A。*

追问：A* 和 Dijkstra 有什么区别?

> 都是最短路搜索，**A* 多了一个启发函数**，会优先往目标方向搜，通常更快；**Dijkstra 是均匀向四周扩**，保证最短但搜得多。我们的地图很小，两者差别不明显，配置选的是 Dijkstra。

Q2-2 全局路径是每次都重算吗? [加分] ★ 容易答错

答： **不是，是条件触发的。** 导航的行为树会**周期性检查**几个条件：路径是不是失效了、目标是不是变了、路径是不是过期了（过期阈值是十几秒量级）。**满足条件才重新规划。** 所以它不是"完全不重规划"，也不是"每秒无条件重算一遍"。无条件重算在算力紧张时会把 CPU 吃满，我一开始就是被这个拖慢的。 **按需重规划**是这里的关键优化。

Q2-3 全局路径规划好之后就一直不变了吗? [加分]

答： **规划好的路径本身不变，但局部会随时调整。** 全局路径是一条**理想参考线**，它只考虑地图上已知的静态障碍；真正开车的是局部规划器，它会**看着实时激光**绕开临时出现的障碍，绕完再回到全局路径上。 这就是为什么要分两层：**全局管大方向，局部管当下怎么开。** 所以界面上看到的那条绿线是**规划路径，不是已经走过的轨迹**，重新规划时它会变——这个区分要能说出来。

Q2-4 目标点的顺序是规划出来的吗？ [加分] ★★ 容易踩坑

答： **不是，顺序是我在配置里写死的。** 五个点按配置顺序走。 **规划出来的只是"点与点之间具体经过哪些位置"**——也就是怎么绕开墙、怎么过门。所以「路线顺序预设」和「自主规划路径」这两件事**同时成立**，不矛盾。 但要说清楚：**这不是求解最优参观顺序的 TSP 系统**，没有做路径总长优化。

技术点 3：局部规划 DWB（5 题）

Q3-1 局部规划器是干什么的？ [必背] ★ 核心

答： 它回答「**这一瞬间该给轮子什么速度**」。它在**局部代价地图**上工作，这一层是跟着机器人移动的一小片窗口。 做法是**采样打分**：生成一批候选速度组合，每个都往前预测一小段轨迹，然后按一堆评价项打分——**会不会撞、离全局路径偏多远、离目标还有多远、速度够不够快**——选总分最高的那个发出去。 这个流程**每个控制周期都跑一遍**，所以能实时避障。

Q3-2 为什么叫「局部」？跟全局的区别是什么？ [必背] ★ 高频

答： **空间范围不同、时间尺度不同、用的地图也不同。** 全局用的是**整张地图**，只在换目标或路径失效时算一次，出的是**一条完整路线**；局部用的是**机器人周围一小块窗口**，每个周期都算，出的是**一条速度指令**。 打个比方：**全局是导航软件给你规划的高速路线，局部是你握着方向盘看着前面实时躲车。** 少了前者会迷路，少了后者会撞车。

Q3-3 采样打分这种方式有什么优缺点？ [加分] ★★

答： 优点是**直观、好调、鲁棒**：不依赖精确的动力学模型，评价项可以随便加减，调权重就能改变行为——想让它更贴墙走还是更保守，改个权重就行。 缺点是**不是最优解**，它只能在你采样的那批候选里挑最好的，采样不够密就可能漏掉更好的轨迹；而且**采样数直接决定计算量**，采样密了算力跟不上。 所以实际调参就是在**"轨迹质量"和"算力"之间找平衡。**我们是把横向采样数压到很小，因为差速车压根不能横移，采了也没用。

Q3-4 差速机器人为什么不能横移？ [加分] ★ 有细节

答： 因为它只有**两个驱动轮**，只能前进后退加转向，没有能侧向推的轮子——像全向轮或者麦克纳姆轮那种底盘才能横移。 数学上就是那两个运动学公式：左右轮同速则直行，有差速则转弯，反向则原地转。**它只有两个自由度：前进速度和转向角速度。** 所以配置里我把横向速度上限设成 0、横向采样数设成 1，**不做无意义的搜索。**这种"知道约束就先把维度砍掉"的做法，是很实在的优化思路。

Q3-5 控制频率怎么定的？调高会怎样？ [加分]

答： 配置的是 10Hz，也就是每秒算十次。 调高的话避障更灵敏、轨迹更平滑，但**计算量线性上涨。**我在只有 4 核的虚拟机上跑，调太高会让整个仿真变卡，反而**实时性下降**——仿真跑得比真实时间还慢，定位和控制全都

跟着劣化。 所以这个值是**对着实际负载试出来的，不是拍脑袋定的。**教训是：**在资源受限的环境里，"更快"常常等于"更慢"。**

技术点 4：代价地图与避障（4 题）

Q4-1 代价地图是什么？跟静态地图有什么区别？ [必背] ★ 高频

答： **静态地图只告诉你哪里有墙**，是建图时一次生成的，永远不变。 **代价地图是在它之上叠加了实时信息的一层"风险图"**：每个格子除了"能不能走"，还有一个**代价值**，越高表示越不想走。它是**动态更新**的——激光一看到新障碍就写进去，障碍移走了再清掉。 规划器找路的时候不是找"能不能走"，而是找**总代价最低的那条**，所以自然就绕开了危险区。

Q4-2 障碍层和膨胀层分别干什么？ [必背] ★★ 简历原话

答： **障碍层**把传感器看到的障碍**标成不可通行**——这是硬约束，直接挡住。 **膨胀层**在障碍周围画一圈**由近到远递减的代价**——这是软约束，能走但"不划算"，越靠近越贵。 膨胀层才是让机器人**愿意离墙远一点**的原因：没有它，规划器会紧贴着墙走，实际轮子稍微偏一点就蹭上了。

追问：那两个半径参数怎么定的？

> 一个是**机器人半径**，近似车身大小，用来判断"这个缝隙钻不钻得过去"；一个是**膨胀半径**，决定离墙多远开始变贵。我是按车身尺寸加安全余量估的，没有做精确的碰撞标定。

Q4-3 全局代价地图和局部代价地图为什么要分成两份？ [加分]

答： 因为**用途和更新频率完全不同。**全局的给规划器找路用，要覆盖整张地图、可以更新慢一点；局部的给控制器避障用，只看车身周围一小块、**必须跟着机器人实时刷新。** 合一份的话会有麻烦：要么全局那张更新太频繁浪费算力，要么局部那张范围太大跟不上实时性。 分成两份之后各自按自己的节奏跑，这是个很典型的**按时间尺度分层**的思路。

Q4-4 动态避障效果怎么样？验证过吗？ [加分] ★ 诚实题

答： **静态绕障验证过了**——场景里的墙和展台都能绕开，一轮五个点全部到达。 但要诚实说：**没有系统验证过动态场景。**比如突然出现一个人挡在前面、或者通道被临时占用，我只是在配置里开了这个能力，**没有做过专门的动态障碍测试。** 另外我**没有测过定位误差**，所以不写"厘米级精度"这种话。能说的是"**在一轮测试中五个目标全部到达**"，这比"成功率 100%"更符合实际证据。

技术点 5：Lifecycle 编排（4 题）

Q5-1 Lifecycle 节点是什么？为什么要它？ [必背] ★★

答： **生命周期节点比普通节点多了几个状态**：没配置 → 已配置未激活 → 已激活，中间还有错误状态。普通节点一启动就干活，**它必须先配置再激活才真正工作。** 要它是因为**导航栈的节点之间有依赖**：地图服务得先把地图加载好，定位才能用；定位和规划都就绪了，行为树才能开始派活。**不编排的话启动顺序一乱就报错。** 用一个生命周期管理器**按顺序统一激活**，是官方给的标准解法。

Q5-2 你是怎么激活的？遇到过问题吗？ [加分] ★★ 最有含金量的坑

答： **写了一个专门的启动激活脚本**，它通过**服务调用**逐个查询节点状态、再发状态切换请求，把那七八个节点按顺序激活。 遇到过问题：**高负载下激活会失败。**四个核心全被仿真和渲染占满，节点响应超时，整个导航栈起不来，后面的导览也就废了。 解决办法是**加重试**——失败就等一会儿再试一轮，还不行就明确报错退出，**绝不带着半激活的状态往下跑。**这个脚本是项目里我写的比较实在的一块工程代码。

追问：还有别的超时问题吗？

> 有，**心跳超时**也会让导航停掉。管理器会跟节点保持心跳，节点响应慢了就判定它挂了。我一方面**降低负载**，另一方面把超时**放宽**到十几秒。代价是**真正故障时发现得也更晚**——这是个明确的取舍。

Q5-3 激活成功就等于导航能用了吗？ [加分] ★ 容易答错

答： **不等于，激活只是必要条件。** 表现是：所有节点都显示已激活，但机器人还是不动。 因为"节点活着"和"数据链路通"是两回事——**命令能不能真的传到轮子上、激光能不能真的进来、坐标变换是不是完整**，激活状态完全不保证这些。 所以我排障时**从不只看激活状态，一定看实际到站日志。**这也是我反复强调的一点：**状态是必要不充分条件，验证要看结果。**

Q5-4 这套导航配置有什么不足？ [加分] ★ 收尾必问

答： 三点。第一，**参数是对着一个固定场景调出来的，泛化性没验证**——换个展厅、加点动态障碍，还得重新调。 第二，**没有做传感器融合**：IMU 有仿真数据也接进来了，但**没接进定位融合**，现在定位完全靠激光加里程计。加 IMU 融合能明显提升抗打滑能力。 第三，**没有系统性的定量评测**：定位误差、避障最小距离、不同场景下的成功率，这些我都没测过数字。要真做产品，这些是必须补的基线。

条目三：导航与导览调度

> **简历原话** 通过 ROS 2 Action 下发导航目标并监听结果回调，使用有限状态机（5Hz 定时轮询）串联迎宾、导航、讲解、问答与巡游切换，处理导航失败与状态回退。
>
> > 🔴 **本条一条专属红线**：导航失败是**跳过当前展点继续巡游，不是回退重试、也不是状态回滚。**简历里的"状态回退"说法要按"跳过并继续"来讲。
>
> **本条四个技术点**：① 状态机怎么实现（6 题） ② Action 下发与结果回调（5 题） ③ 失败处理与人工接管（4 题） ④ 时间与配置（4 题）

技术点 1：状态机怎么实现（6 题）

Q1-1 状态机是怎么实现的？ [必背] ★★ 核心

答： 很朴素：**一个"当前状态"变量 + 一个 5Hz 的定时器。**定时器每 0.2 秒跑一次主循环，循环里根据**当前状态**决定做什么、以及要不要切到下一个状态。 一共九个状态：初始化、空闲、等访客、导航中、讲解中、问答中、切下一站、结束、暂停。 为什么不写成复杂的状态机框架？因为**这个业务的逻辑是线性的**，一个变量加一个 switch 就够清楚了，上框架反而是负担。**工具要匹配问题复杂度。**

Q1-2 为什么用定时轮询，不用事件驱动？ [必背] ★★ 高频

答： **其实是两者结合的。** 外部事件——访客来了、问题来了、人工指令——都是**订阅回调**，是事件驱动；但状态的推进，比如"讲解停留时间到了没"、"问答超时了没"，只能**定时检查。** 用定时轮询的最大好处是**状态极简**：我不需要维护"现在卡在哪一步、在等什么"这种中间状态，每轮进来重新看一眼当前状态和已过时间就够了。代价是**有最多 0.2 秒的延迟**，对导览这种场景完全无感。

Q1-3 5Hz 这个频率是怎么定的？ [加分]

答： **对着业务节奏估的。** 导览里最快的状态切换也就是"导航完成进讲解"，这个粒度是**秒级**的，0.2 秒的轮询精度绰绰有余。 再高没必要——纯属空转烧 CPU；再低会出现"讲解完了还傻等"的延迟感。 而且这个频率是**非常保守的**：定时器回调里只做状态判断和发消息，非常轻，占不了多少资源。**频率的选择，本质上是在"响应延迟"和"空转开销"之间取平衡。**

Q1-4 进入一个新状态的时候，是怎么只播报一次的？ [加分] ★ 细节加分

答： 这是个很实际的细节。因为主循环每 0.2 秒跑一次，**如果不加保护，讲解词会被重复播五次。** 我的做法是**记录进入这个状态的时间戳**，然后用一个**一次性标记**：刚进状态且标记还没置位时才播报，播完立刻置位。 本质上就是**"边沿触发"和"电平触发"的区别**——我要的是状态刚发生变化的那一刻触发一次，而不是每轮都触发。**轮询模型下这个坑很常见。**

Q1-5 状态机一共几个状态？会不会太碎？ [加分]

答： 九个，看着多，但**每个状态职责都很单一**。其中"空闲"和"暂停"是给人工控制用的，"初始化"是等导航服务就绪的。 我觉得不算碎，因为**它们对应的业务动作确实不同**——等访客、走路、讲话、回答问题，行为完全不一样，合并反而会让逻辑变绕。 真要精简的话，"切下一站"这个状态其实可以并进"导航"里，它现在几乎只是做一次索引加一。**但留着它的好处是日志清晰**，一看就知道走到哪一步了。

Q1-6 这个状态机有什么可以改进的？ [加分] ★ 收尾必问

答： 两点。第一，**现在是"等待固定时间"而不是"等待真实完成"**：讲解播完没有、语音念完了没有，我都是靠配一个停留秒数近似。正确做法是让播报节点回一个**完成事件**，改成事件驱动。 第二，**没有状态持久化**：程序一重启就从头开始，不知道上次走到第几个展点。真要产品化，至少要把当前索引和状态存下来，支持**断点续导**。 核心想法是：**用时间近似事件，是能跑但不严谨**，这是我最想改的一处。

技术点 2：Action 下发与结果回调（5 题）

Q2-1 下发一个导航目标，完整流程是怎样的？ [必背] ★ 核心

答： 四步。第一，**构造目标**：从配置里读出这个展点的坐标和朝向，填成一个带坐标系和时间戳的位姿消息。第二，**发目标**：通过 Action 客户端异步发出去，同时挂一个**反馈回调**。 第三，**处理接受**：服务端回来说收下了，但**这只是被接受**；我还要再挂一个**取结果的回调**继续等。 第四，**处理结果**：结果回来了，按状态决定进讲解还是跳过。 一句话：**发目标 → 等接受 → 等结果 → 按结果分支**，中间两步是分开的，不能省。

Q2-2 结果回调是在哪个线程执行的？会阻塞主循环吗？ [加分] ★★

答： 是**异步回调**，由执行器在节点自己的线程里调度，**不会阻塞我发目标的那一句**——发出去就返回了。 但要说清楚：**回调本身还是在同一个执行器上跑的**，所以如果我在回调里做很重的事，主循环照样会被拖住。 我的做法是**回调里只做状态判断和置位**，真正的业务推进交给主循环下一轮去处理。这是 ROS 里很推荐的写法：**回调要短，重活留给主循环。**

Q2-3 导航一直没结果回来怎么办？会卡死吗？ [加分]

答： 理论上**会一直等下去**，我这个项目里**没有给导航单独设超时**。 之所以还能接受，是因为 Nav2 自己有**进度检查**：如果机器人长时间没有靠近目标，它会自己判定失败并返回结果，所以实际上不会真的无限等。 但这是我的**一个明确不足**——业务层没有兜底超时。真要做健壮，应该在状态机里也记一个"进入导航状态多久了"，超时就主动取消目标往下走。**不能把可用性完全寄托在别人的超时上。**

Q2-4 怎么取消一个正在进行的导航？ [加分]

答： **通过 Action 的取消接口**，这是我选 Action 而不是 Topic 的重要原因之一。 代码里我在几个地方会主动取消：收到**停止**指令、收到**暂停**指令、收到**跳过**指令、还有**回到起点**指令。取消之后目标句柄要清掉，避免下一

轮误用。 取消之后结果回调还是会回来的，状态是**已取消**，所以我专门区分了"取消"和"失败"——**取消是人为的，不算异常，不该播报错。**

Q2-5 目标被拒绝了怎么办？ [加分]

答： **被拒绝和失败是两回事，处理方式也不同。** 被拒绝说明服务端**根本没收**——可能是导航服务还没起来。我的处理是**只打一条警告日志，把标志位清掉**，然后主循环下一轮会**重新发一次目标。** 这其实是一个**朴素的重试机制**：因为主循环一直在跑，只要目标没被激活就会重发，所以服务一起来自己就接上了。 失败则是服务端收了但没走到，那是**跳过该展点。**两者不能混。

技术点 3：失败处理与人工接管（4 题）

Q3-1 导航失败了你怎么处理？ [必背] ★★ 核心 + 红线

答： **播报一句"导航出现异常，我将跳过该展点"，然后切到下一站继续巡游。** 判断条件是看结果状态：**成功**进讲解；**被取消**什么都不做（那是人为的）；**其他一律当失败**处理。 这么设计的原因是**展厅导览追求的是整体体验不中断**：一个展区进不去，卡在那儿反复重试会让参观者很难受，不如跳过继续走，把整轮走完。

> 🔴 **红线**：是「**跳过**」，**不是回退重试、不是状态回滚。**别顺着面试官的话头承认"我做了重试"。

Q3-2 支持人工干预吗？ [必背]

答： 支持，通过**一个指令话题**收命令：**开始、停止、暂停、继续、跳过、回到起点**，中英文都认。 比如暂停会**先取消当前导航**再切到暂停状态，继续则从暂停切回导航；回到起点会把索引归零再导航过去。 这个功能很实用：**现场演示的时候随时能接管**，不用重启程序。

Q3-3 一轮走完之后会怎样？能自动循环吗？ [加分] ★ 容易答错

答： **配置里可以开循环**，开的话走完最后一点会回到"等待访客"状态，等下一次触发。 但有个**明确的不足**：**没有自动返航。**机器人仍停在最后一站，离起点的触发位置很远，所以下一轮的迎宾**不会自动触发**，得靠脚本发一条"回家"指令让它先回起点。 这点我不会含糊地说"支持自动循环"——准确说法是**支持循环状态切换，但返航是靠外部脚本触发的。**

Q3-4 暂停之后继续，会从断点恢复吗？ [加分]

答： **不会，是从当前展点重新导航。** 因为暂停的时候我**把导航目标取消了**，继续的时候状态回到导航，主循环发现没有活跃目标，就**重新发一次当前展点的目标。** 好处是**实现简单、状态干净**——不需要保存"走到一半"这种复杂中间状态。 代价是如果已经走了大半，会**退回这个展点重新开始。**更好的做法是暂停时只发零速度让车停住、不取消目标，恢复时继续——但那要求业务层能区分"停住"和"放弃"，当前没做。

技术点 4：时间与配置（4 题）

Q4-1 讲解停留多久、问答等多久，是怎么定的？ [必背]

答： **都是配置项**，写在业务配置里：讲解后停留几秒、问答等待多少秒超时，都能直接改。 问答还有一道**超时兜底**：一直没人提问，超时就播一句"如果没有问题，我们前往下一个展区"，自己往下走。 这个兜底很重要——**不能让流程卡在等人的环节上**。这也是我从"批量操作最怕不知道卡在哪"那条经验里带过来的习惯：**凡是会等的环节，都要有超时。**

Q4-2 仿真时间和真实时间有什么区别？ [加分] ★★ 面试官爱考

答： 仿真里跑的是**仿真时钟**，由仿真器发布。所有节点都配成**使用仿真时间**而不是系统墙上时间。 这么做的意义是：**仿真可以暂停、可以慢放**，暂停时仿真时间也停，所有节点的时间线跟着停，不会乱。 踩过的坑是：**只要有一个节点没配这个开关**，它用的就是真实时间，两边时间对不上，**坐标变换会因为时间戳不匹配直接失败**。这类问题的现象是"数据都有但就是报错"，很难查。

Q4-3 用定时等待代替真实完成，会有什么问题？ [加分] ★ 诚实题

答： **两个方向都会出问题。** 讲解词长的话，**时间没到就播完了**，机器人干等着；讲解词短的话，**还没念完就走了**，会截断。 而且 TTS 的速度是可以配置的，语速一变这个近似就更不准。 正确做法是让播报节点**播完发一个完成事件**，主控收到才继续。我现在是知道差距但没做——**这也是我最想改进的一点。**

Q4-4 展点索引和界面上看到的编号对不上，为什么？ [加分]

答： 因为**代码里的列表索引从 0 开始，可视化标注从 1 开始**，两者差一。 这个坑我在看日志的时候被绕了一次：日志说走到第 0 个点，界面上标的是 1 号。 所以**看日志判断进度时一定要记得加一。**这种"两套编号"的问题在带可视化的系统里很常见，最好在两边都标明清楚。

条目四：建模与仿真

> **简历原话** 编写 URDF/Xacro 描述差速机器人模型，维护 map→odom→base_link→传感器的 TF2 坐标树；在 Gazebo Harmonic 搭建仿真环境，通过 ros_gz_bridge 桥接激光雷达与里程计话题，使用 Xvfb 在无显示服务器跑通 headless 仿真。
>
> **本条四个技术点**：① URDF/Xacro 建模（5 题） ② TF2 坐标树（5 题） ③ Gazebo 与桥接（5 题） ④ 场景与地图生成（5 题）

技术点 1：URDF/Xacro 建模（5 题）

Q1-1 URDF 是什么？里面都描述了什么？ [必背]

答： URDF 是一个 **XML 格式的机器人描述文件**，把机器人拆成**连杆和关节**来描述：**哪块是车体、哪两个是轮子、激光装在哪个位置、它们之间怎么连接。** 每个连杆要写三样东西：**外观**（长什么样）、**碰撞体**（哪些部分会撞到东西）、**惯量**（质量和转动惯量，物理仿真要用）。 有了它，**仿真器、可视化工具、坐标发布节点**都能自动读懂这个机器人的结构，不用各写一份。

Q1-2 Xacro 和 URDF 什么关系？为什么要用它？ [加分]

答： **Xacro 是带宏的 URDF**，先经过一步展开，生成标准的 URDF。 好处就两个字：**复用**。比如左右两个轮子结构完全一样，只是位置镜像，用宏写一次、传不同参数就行；还有惯量计算这种重复公式，也能写成宏。 代价是**多一步转换**，看最终结构得先展开。但对我们这种对称结构的差速车，省下来的重复代码很可观。

Q1-3 外观和碰撞体为什么要分开写？ [加分] ★★ 容易答错

答： 因为**两者的用途完全不同**：**外观**决定你看到什么，可以做得精细；**碰撞体**决定物理引擎算不算撞上，越简单算得越快。 业界通行做法是**碰撞体用简化几何体**——一个长方体代替整个车身。因为碰撞检测每帧都在跑，用精细模型会**严重拖慢仿真**。 我们场景里的展台标记就是**只有外观没有碰撞体**，所以它看得见但挡不住机器人，不会把路堵死。

Q1-4 差速机器人的运动学是什么？ [必背] ★ 八股

答： 就两个公式。**前进速度**等于左右轮速之和的一半乘以轮半径；**转向角速度**等于左右轮速之差乘以轮半径、再除以轮间距。 直观理解：**两轮同速直行，有差速就转弯，反向则原地转。** 在仿真里**不需要我实现这个**——官方的差速插件把速度指令直接转成轮子运动。但**真机上就得自己写或者配驱动**，这是仿真和真机的差别之一。

Q1-5 模型是哪里来的？从头写的吗？ [加分] ★ 诚实题

答： **不是，是拿开源项目改的。** 它已经提供了车体、左右轮、支撑轮和激光的安装位置，我按自己的场景做了适配。 我做的适配主要是：**对接仿真器的插件和话题、调整传感器参数、处理坐标树的冲突。** 这一点必须说清

楚：**底盘结构不是我原创的，适配和集成是我做的。**

技术点 2：TF2 坐标树（5 题）

Q2-1 TF 是什么？为什么需要它？ [必背] ★★ 高频

答： ROS 里用**一棵坐标变换树**来表示各部件之间的相对位置。我们的链是：**地图 → 里程计 → 车身 → 各个传感器。** 需要它是因为**数据都在各自坐标系里产生的**：激光说"正前方 2 米有墙"，但**"正前方"是激光的朝向**，要换算到地图上才有意义。 有了这棵树，**任何两个坐标系之间的转换都能自动算出来**，我不用手写转换矩阵。

Q2-2 map、odom、base_link 三者有什么区别？ [必背] ★★ 必考

答： **map 是地图坐标系**，全局固定，我的展点目标就写在这里。 **odom 是里程计坐标系**，机器人启动时为原点，**连续但有累积漂移。 base_link 是车身坐标系**，跟着机器人动。 关键在 **map 到 odom 这一段**：它由定位模块提供，作用是**修正里程计的漂移。**所以虽然 odom 会飘，但 map 下的位置是准的。 一句话：**map 是"真实在哪"，odom 是"轮子算出来在哪"，两者的差就是定位的修正量。**

Q2-3 每一段变换分别是谁发布的？ [加分] ★ 核心细节

答： **map→odom 由定位模块发**（AMCL）；**odom→base_link 由仿真器的里程计插件发**（真机上是底盘驱动）；**车身到各个传感器的部分由机器人状态发布节点发**，它读 URDF 里写死的安装位置。 这个分工很重要：**每一段只能有一个发布源。** 我踩过的坑就是**两个模块同时发布同一段变换**，导致坐标树冲突、数据乱跳。解决办法是**移除重复的发布源，保证每段唯一。**

Q2-4 坐标变换失败一般是什么原因？ [加分] ★★ 有实战感

答： 最常见的是**时间戳对不上。**ROS 会按时间去查变换，你要查"现在"的变换，但缓存里最新的是几秒前的，超了时间容差就报错。 两个典型诱因：一是**前面说的仿真时间没配统一**，某个节点用真实时间、其他用仿真时间；二是**某个变换压根没发布**，树断了，下游自然查不到。 排查套路是**先打印整棵树，看断在哪一段**，再顺着往上查是谁没发。**按链路查比闷头看代码有效得多。**

Q2-5 这棵树有什么可以改进的？ [加分]

答： 两点。第一，**现在是一棵纯串联的树，没有做传感器融合**：IMU 有数据但没接进定位，所以抗打滑能力弱。加了融合之后，这段变换会更稳。 第二，**没有做时间同步的显式校验**：现在变换失败只能靠日志发现，理想情况应该在启动自检里**检查关键变换是否都存在、时间戳是否正常。** 这也是老问题：**灵活的东西一定要配套检查手段。**

技术点 3：Gazebo 与桥接（5 题）

Q3-1 Gazebo 和 RViz 有什么区别？ [必背] ★★ 高频

答： **Gazebo 是仿真器，RViz 是显示器。** Gazebo 负责**算物理**：重力、碰撞、传感器数据是怎么产生的，它产出的激光是**模拟真实扫描算出来的。** RViz 只负责**把 ROS 里已有的数据画出来**：地图、激光点云、规划路径、机器人模型。它不产生任何数据。 所以**没开 Gazebo 就没有数据可看，没开 RViz 只是看不见**，系统照样跑。这个区分一定要说清楚。

Q3-2 仿真器和 ROS 是两套系统，数据怎么打通的？ [必背]

答： 靠一个**桥接节点**，通过一个**配置文件**声明映射关系：哪个仿真话题转成哪个 ROS 话题、类型是什么、方向是进还是出。 我们桥接的有**时钟、激光、里程计、坐标变换、关节状态、IMU**，以及**反向的速度指令。** 概念上很简单：**两边消息格式不同，中间加一层翻译。** 麻烦的是**类型名和方向必须写对**，写错了的表现是"仿真里有数据但 ROS 收不到"。

Q3-3 桥接最容易踩的坑是什么？ [加分] ★ 有实战感

答： **桥接默认是"懒启动"的：没人订阅就不转。** 这个优化本意是省资源，但会导致**第一个订阅者起来之前，数据一直是空的。** 我遇到过的现象就是：仿真明明在跑、日志也有数据，但 ROS 这边一直接不到。最后是把关键话题的懒启动关掉才解决。 教训是：**这种"按需激活"的优化，一定要知道它存在**，不然排查会完全没有方向。

Q3-4 为什么要 headless？Xvfb 是什么？ [必背]

答： 因为**服务器或者虚拟机没有图形界面**，而 Gazebo 的渲染模块需要显示环境。 **Xvfb 是一个虚拟显示器**，它在内存里模拟一块屏幕，让程序以为自己有显示器可以用，实际不输出到任何物理设备。 我甚至**连 Gazebo 自己的图形界面都不开**，只跑物理和传感器，渲染负担降到最低。这是能在 4 核虚拟机上跑起来的关键优化之一。

Q3-5 在资源受限的机器上，你做了哪些优化？ [必背] ★★ 很能体现工程能力

答： 大概六条。**关掉摄像头**（渲染最吃资源）；**不开仿真图形界面**，用虚拟显示器跑；**降低定位粒子数上限**；**减小局部规划的横向采样数**；**关掉用不上的导航模块**（路径平滑、路点跟随这些）；**给可视化窗口单独限帧和节流时钟。** 还有一个：**给可视化窗口单独把仿真时钟节流到几十赫兹**，导航和仿真本身用原始时钟，互不影响。一句话：**把资源全留给仿真和导航，观察窗口能省则省。**

技术点 4：场景与地图生成（5 题）

Q4-1 展厅场景是怎么搭出来的？ [必背]

答： **用一个 Python 脚本生成，不是手动拖出来的。** 脚本里定义好房间边界、每一面墙的中心和尺寸、四个展台的位置和颜色，然后生成仿真器能读的场景文件。 这么做的好处是**尺寸和坐标完全可控、可复现**——我想改门宽，改一个数字重新生成就行。 **门不是门模型，是墙中间留的一段空隙**，不放碰撞体，机器人就能穿过去。这是个很省事的仿真技巧。

Q4-2 地图是怎么来的？跑 SLAM 建的吗？ [必背] ★★ 红线

答： **不是 SLAM，是脚本按已知几何生成的。** 同一个脚本在生成三维场景的同时，把墙和展台的平面投影**栅格化**成一张二维占据栅格地图，分辨率是每像素 5 厘米。 黑色是障碍、接近白色是可通行、灰色是未知区域。 所以那句话一定要说准：**当前地图是按已知几何预生成的，不是机器人一边走一边扫出来的。**

Q4-3 为什么三维世界和二维地图要用同一套几何来生成？ [加分] ★★ 有设计感

答： 为了避免**对不齐**。如果两边各写一份坐标，稍微有出入，机器人就会觉得"我贴着墙走"，实际在仿真里已经撞上了——表现为定位飘、规划撞墙，而且极难排查。 共用一份几何定义，**从根上消除了这个不一致**。 但要诚实：**共用几何不等于零误差**，栅格化本身有分辨率损失。所以不能说"完全精确"，只能说"**一致性有保证**"。

Q4-4 生成地图的时候要注意什么坐标问题？ [加分] ★ 细节加分

答： **图像的纵坐标方向和世界坐标是反的**——图像的行号向下增加，而世界的 y 轴向上增加。所以生成的时候必须做一次**上下翻转**。 不翻转的表现是：**地图上的墙是倒过来的**，机器人一上来就定位飘。 另外还要在地图文件里写清楚**分辨率和原点坐标**，定位模块靠这两个值把像素换算成米。这两个值错了，整张地图的尺度就是错的。

Q4-5 这套仿真还有什么不足？ [加分] ★ 收尾必问

答： 三点。第一，**没上真机**，真实轮子的打滑、编码器漂移、传感器噪声全都没体现，所以仿真的参数到真机上还得重调。 第二，**没有动态障碍验证**：场景里的东西全是静止的，没测过突然出现的行人或者临时占用通道。 第三，**摄像头一直关着**：视觉这条链路有代码但从未真跑过，相关能力不能算数。 话说回来，**这也是仿真的价值**——把能确定的部分先跑通，把不确定的部分明确标出来。

条目五：访客检测与迎宾

> **简历原话** 基于视觉 marker 与距离阈值判定访客到位并触发迎宾播报，检测距离等阈值参数化可配。
>
> > 🔴 **本条红线最高**：**默认是"机器人走到预设点附近、距离小于阈值"触发，不是摄像头、不是识别人。** 视觉分支默认关、没验证过。面试照这句答，别顺着话头说成"视觉识别访客"。
>
> **本条三个技术点**：① 距离触发怎么做的（4 题） ② 视觉检测分支（4 题） ③ 边界与诚实（3 题）

技术点 1：距离触发怎么做的（4 题）

Q1-1 迎宾是怎么触发的？ [必背] ★★ 核心 + 红线

答： **默认是一个距离判断。** 我在配置里给了一个"访客标记点"的坐标，机器人一边走一边算**自己到这个点的距离**，小于阈值（一米多）就触发一次迎宾，往话题上发一个事件。 主控收到这个事件，播一句"您好，检测到您的到来"，然后开始导览。 所以要说清楚：**它判的是"机器人有没有走到那个位置"，不是"有没有看到一个人"。**

Q1-2 为什么要做成距离触发？ [加分]

答： 因为**个人条件下没有可靠的视觉方案**，而我要先跑通的是**上层的业务流程。** 这是个很典型的**先解耦后替换**的思路：**把"访客到位"抽象成一个事件**，至于这个事件是距离算出来的、摄像头看出来的、还是红外感应出来的，**对主控来说完全一样。** 将来接真实传感器，只需要换掉这一个节点，主控一行不改。**接口定好了，实现可以后面补。**

Q1-3 会不会重复触发？ [加分] ★ 容易答错

答： **做了两道防重复。** 第一道在检测节点里：触发一次之后置一个**已检测标记**，不再重复发。 第二道在状态上：检测节点**订阅了主控广播的状态**，**只有在"等待访客"这个状态下才检测**；一旦离开这个状态，标记自动复位，等下一轮。 这个**"靠状态同步来复位"的设计挺好用：检测节点不需要知道导览进行到哪一步，只看当前状态就行。**

Q1-4 检测频率是多少？为什么不是每帧？ [加分]

答： **两秒一次**，用定时器跑，不是订阅里程计每条消息都算。 因为机器人移动速度很慢，**每帧算纯属浪费。**而且距离判断对实时性要求极低——晚个一秒触发，参观者完全无感。 这个思路跟前面"上报频率要匹配被观测量的变化频率"是同一条：**慢变量就用慢频率。**

技术点 2：视觉检测分支（4 题）

Q2-1 你说还有一套摄像头检测？它做了什么？ [必背] ★ 诚实题

答： 有这个分支，但**默认关闭、也没验证过。**它的做法是：把图像转成 **HSV 色彩空间**，取**红色区间**做掩膜，统计红色像素个数，超过阈值（几百像素）就认为"看到了红色物体"。 之所以取两段红色区间，是因为**红色在 HSV 里跨了 0 度和 180 度两头**，必须并起来才算全。 **它是"检测红色物体"，不是"识别访客"。** 场景里那个红色立柱就是干这个用的。

Q2-2 为什么用 HSV 不用 RGB? [加分] ★★ 八股

答： 因为 **RGB 三个通道都受亮度影响**，光照一变，同一个红色的 RGB 值就变了，阈值很难调。 **HSV 把亮度单独拆出去了**：H 是色相、S 是饱和度、V 是明度。我只在 H 和 S 上设阈值，**亮度变化基本不影响结果。** 这是做颜色检测的标准做法：**先换到把亮度和颜色分离的空間，再做阈值。**

Q2-3 装不上依赖会怎样? [加分] ★ 有工程意识

答： **会自动降级。** 视觉那套需要图像转换库和 OpenCV，我在初始化的时候**用异常捕获包住**，装不上就**打一条警告日志，自动退回距离触发模式。** 好处是**不会因为一个可选依赖缺失就让整个节点起不来。** 这个习惯我在别的地方也用：**可选能力要能优雅退化，不能让短板拖垮主流程。**

Q2-4 视觉检测有什么明显不足? [加分]

答： 两个致命问题。第一，**红色就触发太粗糙**：停在场景里的红色立柱会一直触发，穿红衣服的人也会被当成访客，**误报率会很高。** 第二，**光照敏感**：虽然用了 HSV，但饱和度和明度阈值还是得对着实际光照调，换个展厅就得重新调。 所以我对它的定位很清楚：**是一个预留的扩展点，不是已经验证的能力。** 真要做访客识别，得上人体检测或者人脸检测模型。

技术点 3：边界与诚实（3 题）

Q3-1 阈值参数是怎么管理的? [必背]

答： **全部做成节点参数**，通过启动文件传进去：标记点坐标、距离阈值、红色像素阈值，都能在启动命令里覆盖。 好处是**不用改代码就能调**，而且**两套模式的参数是分开的**，切模式只改一个开关。 这个**"凡是阈值都参数化"的习惯是我在别的项目里养成的：现场要调的东西，一定要能在外面调。**

Q3-2 那这个"访客检测"到底算不算真做了? [必背] ★★ 保底诚实题

答： **要分两层说。** 做了的是：**触发逻辑、防重复、状态同步、参数化、以及跟主控的事件衔接**——这一整套链路是完整的、跑通的。 没做的是：**真实的访客感知。**默认模式是位置判断，视觉分支默认关且未验证。 所以准

确说法是：**我实现了"访客到位事件"这条链路，但感知手段用的是仿真替代。** 这个边界我不会含糊。

Q3-3 要变成真能用的访客检测，你会怎么做？ [加分]

答： 三步。第一，**换感知手段**：用人体检测模型替代颜色阈值，或者干脆用超声波/红外这种更便宜的方案判断"有人靠近"。 第二，**加确认机制**：检测一次不算数，**连续几帧都检测到才算真的有人**，避免误触发。 第三，**加超时和重试**：等太久没人来就播一句提示或者回待机，别一直傻等。 核心是：**把"检测到"从一个瞬时判断，变成一个带确认的状态。**

条目六：语音播报与问答

> **简历原话** 对接离线 TTS 合成讲解音频并按展点播报；实现基于关键词匹配的问答节点，问答规则与展点讲解词由 YAML 配置驱动，便于扩展新展点。
>
> > 🔴 **本条两条红线**：① 问答是**关键词规则匹配，不是大模型、不是语义理解**；② 语音输入是**模拟脚本，没有真实语音识别。**
>
> **本条三个技术点**：① TTS 播报（4 题） ② 关键词问答（5 题） ③ 模拟语音输入（3 题）

技术点 1：TTS 播报（4 题）

Q1-1 语音播报是怎么做的？ [必背]

答： 用的是**本机离线的语音合成引擎**，收到文字就合成发声，**不联网、不调任何云端接口。** 架构上它是一个**独立节点**：订阅一个"要说的话"的话题，收到就播。主控只负责把讲解词发到这个话题上，**完全不知道声音是怎么出来的。** 好处是：**换引擎、或者干脆换成文字输出，只改这一个节点。**

Q1-2 为什么用离线引擎？ [加分]

答： 三个原因。第一是**延迟低**，本地合成几乎立刻出声，云端要往返一趟；第二是**不依赖网络**，展厅断网也能讲；第三是**没有调用成本和隐私问题。** 代价是**音质不如云端方案**，机械感重一些。 对导览这种**内容固定、句子短**的场景，离线的劣势几乎不影响体验，优势却很明显。所以选型是**看场景权衡，不是追新。**

Q1-3 如果没有音频设备怎么办？ [加分] ★ 有工程意识

答： **自动降级成文字输出。** 我在初始化的时候捕获异常，引擎起不来就把标志位置掉，之后收到文字**直接打到日志和终端。** 好处是**在没有声卡的虚拟机、服务器上也能完整验证整条链路**——消息到了没有、内容对不对，看日志就知道。 这一点很重要：**降级不是为了省事，是为了让流程在任何环境下都能被验证。** 我在开发阶段大部分时间就是靠文字输出调的。

Q1-4 播报为什么单独一个节点？合进主控不行吗？ [加分]

答： **不行，因为合成播放是阻塞的。** 引擎在念一句话的时候会把调用的线程占住，如果合进主控，**主循环就停了**，状态机这几十秒什么都干不了。 独立成节点之后，**主控发完消息立刻返回**，播报在另一个进程里慢慢播，互不干扰。 这是个很实在的理由：**拆节点的判据之一就是"这个操作会不会阻塞别人"。**

技术点 2：关键词问答（5 题）

Q2-1 问答是怎么实现的？ [必背] ★★ 核心 + 红线

答： **关键词规则匹配。** 配置文件里是一张规则表，每条规则有**一组关键词和一个答案**。收到问题后，把问题和关键词都转成小写，**逐条遍历，看关键词是不是出现在问题里**，第一条命中的就返回它的答案。 一条都没命中，就返回**配置里那个默认的兜底回答。** 所以它是**规则匹配，不是语义理解。**问法变了但关键词还在，就能答；换个说法关键词对不上，就答不了。

Q2-2 为什么要转小写再匹配？ [加分]

答： 为了**消除大小写的干扰。**中文场景其实影响不大，但如果问题是英文或者中英混排，"ROS" 和 "ros" 应该算同一个词。 标准做法是**统一归一化后再比较**——转小写是最简单的一层。更完整的话还要去掉标点符号、多余空格。 这是个很小但很能说明习惯的细节：**字符串匹配前先归一化。**

Q2-3 规则的优先级是怎么定的？ [加分] ★ 容易答错

答： **按配置里的顺序，越靠前优先级越高**，命中第一条就返回，**后面的不再看。** 所以**规则的顺序是有意义的**：越具体、越容易误命中的关键词，应该放前面。比如"便民服务"这种明确的词组要放在"服务"这种泛词前面。 这是个隐性约束——**配置文件看着是数据，其实顺序承载了逻辑。**这类"约定式配置"的问题我前面也提到过：**没有强约束，容易被人改坏。**

Q2-4 加一条问答要改什么？ [必背] ★ 高频场景题

答： **只改一个规则配置文件**，加一条关键词加答案，**代码一行不动。** 问答节点在**启动时**把这张表读进来，所以改完要**重启这个节点**才生效。 这也是简历上那句"便于扩展新展点"的落点：**加展点是加配置，加问答也是加配置，业务代码零改动。**

Q2-5 这个问答有什么明显不足？换你会怎么改？ [加分] ★ 收尾必问

答： 三个不足。第一，**只能关键词命中**，换个说法就答不了，而且**完全不理解上下文**——你问"它有什么"，它不知道"它"是谁。 第二，**没有得分机制**：现在是谁先命中就用谁，更好的做法应该是**统计命中了几个关键词、按得分排序取最高的**，准确率会明显提升。 第三，**规则多了会有冲突**：现在靠人排顺序，规则一多就维护不动了。 真要升级，我倾向于**接一个本地小模型做语义匹配**，规则表退化成兜底和高频问题的快速通道——**规则管准确，模型管泛化。**

技术点 3：模拟语音输入（3 题）

Q3-1 语音输入是真的吗？ [必背] ★★ 红线

答： **不是，是模拟的，没有做真实的语音识别。** 它是一个脚本：**订阅主控广播的状态，一旦发现进入"问答"状态，等几秒就从预置的问题列表里随机挑一条发到问题话题上。** 这么做的目的很单纯：**让整个演示全程不需要人干预**，在没有麦克风的虚拟机上也能自动跑完一轮。

Q3-2 模拟输入也会防重复吗？ [加分]

答： 会。它记录**本次问答是否已经提问过**，进入问答状态时复位，**提问后置位**，保证一个展点只问一次。 而且它也是**看状态复位**的，跟迎宾检测是同一个套路：**靠主控广播的状态来同步节奏，节点之间不需要互相认识。** 这个模式我在项目里用了三次，很好用：**状态广播出来，谁关心谁订阅。**

Q3-3 要接真实的语音识别，你会怎么改？ [加分]

答： **架构上不用改，换掉一个节点就行。** 因为"问题"已经是一个标准的消息了，主控和问答节点都只认这个消息，**不关心它是哪来的。** 要改的是**这一层内部的工程问题**：**语音端点检测**（什么时候算说完了）、**降噪**、**识别置信度低的时候要不要追问**，还有**识别延迟**——人说完到出结果有一两秒，问答的等待超时得放宽。 所以我才说**接口抽象的价值在后面**：当时多做一层，后面换实现就只是换一个节点的事。

附录：通用追问（6 题）

Q1 这个项目你具体做了什么？哪些是别人的？ [必背] ★★ 最重要的一题

答： 我做的：**展厅场景和地图的生成脚本、机器人模型的适配、Nav2 导航栈的集成和调参、导览状态机、五个业务节点的拆分和通信设计、迎宾检测、语音播报和问答节点，还有启动脚本和可视化工具。** 不是我做的：**导航算法本身**（Nav2 是开源栈，我是集成和调参）、**仿真器**（Gazebo 是官方的）、**底盘模型**（开源项目改的）。 还有三块是**明确没做的**：**没上真机、没跑 SLAM 建图、没有真实语音识别和视觉访客识别。**

> 💡 打分点在**边界清晰**。说"我集成和调参了 X，自己写了 Y，Z 没做"，比"这个项目我都接触过"靠谱得多。

Q2 你遇到过最难的调试是什么？怎么定位的？ [加分] ★

答： 印象最深的是**"目标被接受但机器人不动"**。现象是导航服务起来了、目标也接受成功了，但轮子纹丝不动，日志还没有明显报错。 我的排查是**顺着链路一段一段查**：先看控制器有没有输出速度 → 再看平滑器有没有转发 → 再看桥接有没有收到 → 最后看仿真里的底盘插件有没有响应。最后定位到**桥接这一段的数据没通。** 教训是：**遇到"看起来都正常但不工作"的问题，不要闷头看代码，把链路画出来一段一段验。**还有一条：**"激活成功"是必要条件不是充分条件，验证要看结果。**

Q3 为什么要选 ROS 2 而不是自己写通信？ [必背]

答： 因为**我要解决的问题是导览业务，不是通信本身。**ROS 2 把节点发现、消息序列化、发布订阅、服务、动作、坐标变换、参数、启动编排**全都有现成实现**，我直接用就行。 自己写的话，光是跨进程通信和坐标变换就得花掉大半时间，而且**肯定没有它稳定。** 代价是**引入了一个很重的学习曲线和依赖**，还有**中间件的性能开销。**但对这个场景完全划算——**站在成熟框架上，才能把精力放在业务上。**

追问：ROS 2 和 ROS 1 有什么区别？

> 最大的变化是**通信底层从自己实现的 TCP/UDP 换成了 DDS 标准**，带来了去中心化发现、更好的实时性和服务质量策略。另外就是**节点生命周期管理、Action 的改进、以及更好的多机器人支持。**我只用过 ROS 2，ROS 1 没实际项目经验。

Q4 你在这个项目里踩过哪些印象深的坑？ [必背] ★ 必备故事

答（挑三个讲，每个都要有"现象 → 排查 → 原因 → 解决"）： **第一个是坐标变换冲突的坑。** 现象是机器人位置乱跳、定位飘。原因是**两个模块同时发布了同一段坐标变换**，树出现了两个父节点。解决办法是**移除重复发布源，保证每段变换只有一个来源。**

第二个是"数据都有但收不到"的坑。 现象是仿真日志里有激光数据，但 ROS 这边订阅是空的。原因是**桥接默认懒启动**——没有订阅者的时候它根本不转。解决办法是**把关键话题的懒启动关掉。**

第三个是高负载下激活失败的坑。 现象是导航栈起不来，日志报超时。原因是**4 核虚拟机被仿真和渲染占满**，节点响应超时。解决办法是**激活加重试 + 放宽心跳超时 + 全力降负载。**

还有个小坑：改完配置发现没生效，折腾半天才想起来**配置是启动时读的，得重启节点。**

> 💡 讲坑一定要讲**你怎么查出来的**和**吸取了什么教训**，不要只讲现象。

Q5 你觉得自己这个项目有哪些做得不够好的地方？ [加分] ★ 必问收尾

答： 坦诚说有四点。第一**没上真机**，这是最大的局限——真实噪声、打滑、传感器误差全没体现，参数到真机上得重调。 第二**用固定等待代替真实完成**：讲解完没完、话念完没完，都是靠配秒数近似，不严谨。 第三**可观测性不够**：节点活着没有、导航卡在哪一步，主要靠翻日志，应该做心跳和自动拉起。 第四**验证不够系统**：定位误差、避障最小距离、动态场景成功率，这些数字我都没测过，所以只能说"一轮测试中五个目标全部到达"，不敢说"成功率 100%"。

> 💡 主动承认问题 + 给出改进方案 = 有反思能力。比"我觉得挺好的"强一百倍。

Q6 你从这个项目里学到了什么？ [必背]

答： 三个层面。**技术层面**：把机器人这一套完整走了一遍——建模、坐标变换、仿真、定位、规划、控制、业务调度，知道了每个环节解决什么问题、谁依赖谁。 **设计层面**：学会了**用接口隔离变化。**迎宾可以是距离也可以是视觉，语音可以是模拟也可以是真实识别，问答可以是规则也可以是模型——**只要接口定好，换实现就是换一个节点的事。**最大体会是：**好的设计不是让它现在好写，而是让它以后好改。** **工程层面**：调试能力提升最多——**按链路分段排查、用日志验证而不是靠看画面、资源受限时先砍掉非必要负载。**还有一条：**主动把边界说清楚，比被追问出来强得多。**

附录二：简历名词八股（口语背诵版）

> **这部分补什么**：简历「专业技能」里出现的技术名词，面试官可能**脱离项目**直接问"这是什么 / 和那个什么区别"。跟前面不一样——前面是"你项目怎么做的"，这里是"这个技术点本身你懂多少"。
>
> **背题优先级**：**ROS2 通信底层** 和 **C++ 语言** 最容易被问（因为简历明写"熟悉 C++""会用 ROS2"）；**网络并发** 你简历写了但**没有项目直接支撑**，最危险，背之前先看每题的「📌 诚实边界」。
>
> **答题姿势**：名词题三件套——**一句话定义 → 核心机制 → 为什么用它（对比）**。念出口 3~5 句。

技术点 1：ROS2 通信底层（DDS / QoS / 回调）（8 题）

Q1-1 ROS2 的通信底层是什么？跟 ROS1 有什么区别？ [必背] ★★ 高频

答： ROS2 底层用的是一个叫 **DDS 的通信标准**，ROS1 是自己在 TCP/UDP 上实现的通信。 DDS 是个通用中间件协议，ROS2 把节点发现、消息序列化、发布订阅这些**全交给 DDS 去做**。 换成 DDS 带来的变化是：**去中心化**——没有中心节点，节点之间自动发现彼此；**实时性更好**；还能配 **QoS 服务质量**策略。 代价是引入了新依赖、也更复杂。**一句话：通信从"自研"换成了"行业标准"。**

追问：DDS 是公司还是协议？

> DDS 是**协议标准**，不是某家公司。它是一套规范，有多个厂商的实现（FastDDS、CycloneDDS 等），ROS2 默认用的是 FastDDS。

Q1-2 QoS 是什么？有哪些常用策略？ [必背] ★★ 八股

答： QoS 是**服务质量策略**，用来约定"这条消息怎么传、传丢了怎么办"。 最常用的几个：**可靠性**（Reliable 保证送到 / BestEffort 尽力而为，丢了不管）；**历史深度**（存几条历史消息）；**持久性**（新的订阅者能不能拿到之前发的）。 不同的场景选不同的组合：**传感器数据**一般是 BestEffort——丢了就等下一帧，追历史没意义；**关键指令**用 Reliable——必须送到。 这是个很实际的权衡：**可靠和实时常常要二选一。**

追问：你项目里用过 QoS 吗？

> 我没显式去配 QoS，用的默认值。但我理解这个机制——比如导航的关键消息就该可靠，点云这种高频数据就该尽力而为。

Q1-3 回调是在哪个线程执行的？多线程要注意什么？ [必背] ★★ 核心

答： 回调由 **executor（执行器）** 调度。默认是**单线程**执行器，所有回调**串行**跑在一个线程里——一个回调没跑完，其他回调都得等着。 也可以配**多线程执行器**，回调就能并行跑，但代价是**要自己保证线程安全**：如果多个回调改同一份数据，就必须加锁。 我的做法是**故意用单线程**：回调里只做"把数据存下来"这一件事，真正的处理放到主循环里串行做，**从根上避免并发问题。回调要短，重活留给主循环。**

Q1-4 节点之间怎么发现彼此的？需要配 IP 吗？ [加分]

答： **不需要手动配，靠 DDS 的自动发现机制。** 节点启动后会在局域网里广播自己的信息，其他节点收到后自动建立通信。 所以 ROS2 里**没有中心注册表**，这就是前面说的"去中心化"。 代价是**跨机器通信要网络能通**（同一网段、端口不挡），而且发现有一个过程，不是瞬间的——所以节点之间**第一次通信前要等发现完成。**

Q1-5 Protobuf 是什么？和 ROS 消息有什么关系？ [加分] ★ 简历有

答： Protobuf 是一种**结构化数据的序列化格式**，用来把对象变成字节流传输，定义消息用 `.proto` 文件。ROS 的消息也有自己的格式（`.msg` 文件）和序列化机制，**思路是相通的**：先定义结构，再自动生成编解码代码。 区别是 **Protobuf 是通用的**，不依赖 ROS，任何系统都能用；ROS 消息是 ROS 生态自带的。 我简历里 Protobuf 主要用在**上一个实习项目**（路侧感知），那边用它封装视频流消息，跟这个机器人项目是两套。

Q1-6 序列化是什么？为什么要序列化？ [加分]

答： 序列化就是**把内存里的对象变成一串字节**，反序列化就是变回来。 为什么需要：因为**跨进程、跨机器、跨语言**传输，对方不知道你的内存长什么样，只能传字节。 所以约定一个格式（怎么编码、字段怎么排），两边都按这个格式来，就能还原。 代价是**有编解码开销**，传大消息（比如图像）时这个开销很明显，所以会有共享内存、零拷贝这类优化。

Q1-7 发布订阅和请求响应各自的适用场景？ [加分]

答： **发布订阅（Topic）适合"一对多、持续、不关心结果"**——比如传感器数据，谁要谁订阅，发出去不用等回执。 **请求响应（Service）适合"一问一答、要结果"**——比如查个状态、要个配置。 **Action 是两者的升级**——适合"长任务、要反馈、要能取消"，比如导航。 判断标准很简单：**看你要不要结果、要不要中途取消、是不是长任务。**

Q1-8 说说你对 ROS2 节点、话题、消息这三个概念的理解？ [必背]

答： **节点**是运行的基本单位，一个进程，负责一件具体的事；**话题**是节点之间通信的"频道名"，发布者和订阅者通过它对接；**消息**是传的数据本身，有固定的类型。 打个比方：**节点是人，话题是频道，消息是说的话。** 一个人往某个频道喊话，订阅这个频道的人都听得到，但**谁在听、有几个人听，说话的人不知道。** 这套模型的精髓是**解耦**：加一个节点、减一个节点，都不用改别人。

技术点 2：C++ 语言八股（8 题）

> 📌 **诚实边界**：简历写"熟悉 C++"，但**这个机器人项目主代码是 Python**，C++ 主要体现在上一个实习（路侧感知 AirOS）和你简历的通用技能。被问 C++ 时，**能答原理、但别声称"这个项目是 C++ 写的"。**

Q2-1 RAII 是什么? [必背] ★★ 简历原话

答： RAII 是**"资源获取即初始化"**，核心思想是：**把资源的生命周期绑在对象的生命周期上。** 在构造函数里拿到资源（比如 new 一块内存、打开一个文件），在**析构函数里自动释放。**这样对象销毁时，资源**一定**会被释放，不管中间有没有异常、有没有提前 return。 它解决的是 C++ 里最头疼的问题——**内存泄漏和资源没释放。** 典型应用就是智能指针、文件句柄、锁。**一句话：让资源"用 RAII 对象管着，自动生、自动死"。**

追问：没有 RAII 会怎样?

> 就得**手动配对** new/delete、open/close。一旦中间有异常或者提前返回，某个 delete 没执行到，就泄漏了。RAII 把这些"别忘了关"变成"自动关"。

Q2-2 智能指针有哪几种? 区别是什么? [必背] ★★ 高频

答： 三种。**unique_ptr** 独占所有权，**不能被复制**，只能移动——一个资源同一时刻只有一个主人。**shared_ptr** 共享所有权，内部**引用计数**，最后一个持有者销毁时才真正释放。 **weak_ptr** 不参与计数，专门用来**打破 shared_ptr 的循环引用**，防止内存泄漏。 选型：**能独占就用 unique_ptr**（最轻、零开销），确实要共享才用 shared_ptr，weak_ptr 只在解决循环引用时出场。

追问：shared_ptr 的引用计数线程安全吗?

> **计数本身是原子操作，线程安全的**；但它管理的**对象**不是——多线程同时改同一个对象还是得自己加锁。

Q2-3 移动语义是什么? 解决了什么问题? [必背] ★ 简历原话

答： 移动语义是**把资源"偷"过来而不是复制。**通过右值引用和移动构造函数，把一个对象的资源**直接转移**给另一个对象，原来的对象变成空壳。 它解决的是**深拷贝太贵**的问题：比如一个装着大数组的对象，复制要复制整个数组，移动只需要**把指针换过去。** 典型场景是**返回值优化、容器扩容、unique_ptr 的传递。** 一句话：**复制是"再抄一份"，移动是"把东西搬走"。**

Q2-4 多态是怎么实现的? 虚函数的作用? [必背] ★ 简历原话

答： 多态是**"同一个接口，不同对象有不同行为"。实现靠虚函数 + 虚函数表**：基类里声明虚函数，子类重写，运行时通过虚表查到**实际该调哪个子类的版本。** 好处是**面向接口编程**：上层代码只认基类指针，不关心具体是哪个子类，加新子类不用改上层。 代价是**有虚表查询的间接开销**，以及对象里多一个虚表指针。 **一句话：多态让代码"对扩展开放、对修改封闭"。**

Q2-5 STL 容器里 vector 和 list 的区别? 什么时候用哪个? [必背] ★ 八股

答： **vector 是连续内存的动态数组**，支持**随机访问**（O(1) 下标），但中间插入删除要移动后面元素（O(n)）。 **list 是双向链表**，**头尾和中间插入删除都是 O(1)**，但不能随机访问，只能遍历。 所以：**查得多、顺序存 → vector**；**频繁在中间插删 → list。** 实际工程里 **vector 是默认选择**，因为缓存友好、连续内存遍历快，list 用得相对少。

Q2-6 C++ 里 new/delete 和 malloc/free 有什么区别? [加分]

答： 最大的区别：**new 会调用构造函数，delete 会调用析构函数**，malloc/free 只分配/释放内存，**不碰对象的构造和析构。** 还有：new 是**运算符**（能重载），malloc 是**库函数**；new 失败抛异常，malloc 失败返回空指针；new 能算类型大小，malloc 要手动写字节数。 C++ 里应该优先用 new，更要优先用**智能指针代替裸 new**。malloc 主要是 C 的习惯。

Q2-7 引用和指针有什么区别? [必背] ★ 基础高频

答： 三点。第一，**引用必须初始化**，指针可以空着；第二，**引用不能改指向**，一旦绑定一个对象就绑死了，指针能重新指向别处；第三，**引用没有空引用**，指针有空指针。 语法上引用用起来更像别名，不需要解引用。 所以**能确定不会空的场景用引用**（比如函数参数、遍历），**可能为空、可能要改变指向的用指针。**

Q2-8 多线程里加锁要注意什么? 死锁怎么避免? [加分]

答： 加锁要注意：**锁的粒度要小**（只锁必要的那几行，别整个函数都锁）；**要成对**（lock 了必须 unlock，用 RAII 的 lock_guard 最安全）。 死锁最常见的原因是**两个线程按相反顺序拿两把锁**——一个先拿 A 再拿 B，另一个先拿 B 再拿 A，就互相卡死。 避免办法：**固定加锁顺序**（大家永远按 A→B 的顺序拿）、**尽量只拿一把锁**、**用 try_lock 加超时。** 一句话：**锁要少、要短、要按固定顺序。**

技术点 3：Python 语言八股（4 题）

> 📌 简历写"会用 Python"，而这个项目**主代码就是 Python**，所以 Python 八股**反而是最该答好的**——你有真项目背书。

Q3-1 Python 的 GIL 是什么? [加分] ★★ 高频

答： GIL 是**全局解释器锁**，它保证同一时刻**只有一个线程在执行 Python 字节码。** 所以 Python 的多线程**在 CPU 密集任务上并不能真正并行**——多线程只是快速切换，不是真并行。 但**IO 密集任务**（网络、文件、等数据）不受影响，因为等 IO 的时候会释放 GIL，别的线程能跑。 所以：**算得多 → 用多进程**；**等得多 → 用多线程。** 我们项目里的节点是**多进程**（ROS2 每个节点一个进程），天然避开了 GIL 的问题。

Q3-2 装饰器是什么? 你用过吗? [加分]

答： 装饰器是一个函数，**接收一个函数、返回一个函数**，用来在不改原函数代码的情况下**给它加功能。** 用 `@装饰器名` 写在函数上面就行。 典型用途：日志、计时、权限校验、缓存。 它本质是**高阶函数的语法糖**——"给函数包一层"。我理解这个机制，项目里没有特别复杂的用法。

Q3-3 生成器是什么？有什么好处？ [加分]

答： 生成器是**一种惰性求值的迭代器**，函数里用 `yield` 代替 `return`，调用时**不是一次性算出所有结果，而是用一次算一个。** 好处是**省内存**：处理大数据、大文件时，不用把整个结果都装进内存。 比如读一个超大的日志文件，用生成器一行一行吐出来，内存占用是常数级的。 **一句话：用的时候才生产，不用不占地方。**

Q3-4 深拷贝和浅拷贝的区别？ [加分] ★ 基础

答： 浅拷贝只复制**最外层**，里面的可变对象（比如列表里的列表）**还是共享的**——改一个会影响到另一个。 深拷贝会**递归复制所有层**，两个对象完全独立。 Python 里 `copy.copy()` 是浅拷贝，`copy.deepcopy()` 是深拷贝。 坑通常出在**浅拷贝共享了可变对象**——你以为复制了一份，结果改一边另一边也跟着变。

技术点 4：网络与并发八股（6 题）

> 🔴 **诚实边界（先看这条）**：简历写"用 epoll/Reactor 写过主从式高并发网络服务""实现生产者-消费者"，但**这个机器人项目里没有这些**——它们来自你之前的项目/学习。 被问到时，**说原理没问题，但要说清"这是在之前 XX 里做的"**，别让面试官以为机器人项目里有高并发网络服务。如果之前那个项目你已经吃透了，就把场景补上；如果只是学过写过 demo，就诚实说"写过 demo / 学习项目"。

Q4-1 epoll 是什么？和 select 有什么区别？ [加分] ★ 简历原话

答： epoll 是 Linux 上的**IO 多路复用**机制，用来**一个线程同时监听很多个连接**，哪个有数据了才处理哪个。 和老的 select 比，核心优势是：**select 要遍历所有连接去查"谁有数据"，连接一多就慢**；epoll 是**事件驱动**的，内核直接告诉你有几个连接就绪了，**不用遍历。** 所以 **epoll 在连接数很大时性能远好于 select。** 这也是高并发网络服务器（比如 Nginx）都用 epoll 的原因。

追问：边缘触发和水平触发什么区别？

> **水平触发**：只要还有数据没读完，就**一直通知**你；**边缘触发**：只在数据**从无到有**那一瞬间通知一次，之后必须一次性读完。边缘触发更高效但要小心处理，简历里写的是边缘触发。

Q4-2 Reactor 模式是什么？ [加分] ★ 简历原话

答： Reactor 是一种**事件驱动的网络编程模式**：一个主线程负责**监听事件、分发事件**，把就绪的连接交给工作线程去处理。 核心就一个循环：**等事件 → 分发 → 处理**，不停地转。 好处是**一个线程能管很多连接**，不用为每个连接开一个线程（那是老的一连接一线程模型，连接一多就扛不住）。 它和 epoll 是搭配的：**epoll 负责"发现谁有事件"，Reactor 负责"怎么组织这些事件的处理"。**

Q4-3 生产者-消费者模型是什么？怎么实现的？ [加分] ★ 简历原话

答： **生产者**负责生产数据放进一个**队列**，**消费者**从队列里取数据处理，两者**通过队列解耦**，速度不一致也能各自跑。 关键同步手段是：**互斥锁**保护队列不被同时读写，**条件变量**用来在"队列空了消费者等待""队列满了生产

者等待"之间通知。 这个模式在项目里很常见——比如 ROS2 内部的消息队列本质就是生产者消费者。 **一句话：一个缓冲 + 两把同步，让快慢两头各跑各的。**

Q4-4 线程池是什么？为什么用？ [加分] ★ 简历原话

答： 线程池是**预先创建一批线程**放在池子里，来了任务就交给空闲线程执行，执行完线程回收回池子，而不是用完就销毁。 好处是**省掉频繁创建/销毁线程的开销**——线程创建销毁很贵，池子里的线程复用就便宜。 还顺带**控制了并发度**——池子多大，最多就多少个任务同时跑，不会无限开线程把资源打爆。 缺点是要管理任务队列和线程的分配。**一句话：线程池是用空间换"创建线程的时间"。**

Q4-5 TCP 和 UDP 的区别？ [必背] ★★ 基础高频

答： **TCP 是面向连接、可靠的**：先三次握手建立连接，数据**保证按序、不丢、不重复**地送达，有确认和重传机制，代价是慢、开销大。 **UDP 是无连接、不可靠的**：直接发数据报，**不保证到达、不保证顺序**，但快、开销小。 选型：**文件传输、网页这种不能丢的 → TCP**；**视频直播、游戏、实时传感器这种丢了无所谓、要快的 → UDP。** ROS2 底层的 DDS 里，**可靠的 QoS 类似 TCP，尽力而为的类似 UDP** 的思路。

Q4-6 多进程和多线程怎么选？ [加分]

答： 核心看**任务类型和隔离需求。 多线程**共享内存、通信快、创建开销小，但**一个线程崩可能带崩整个进程**，还要处理锁的问题。 **多进程**内存隔离、一个进程崩不影响别的、天然并行，但**进程间通信和启动开销更大。 CPU 密集 → 多进程**（尤其在 Python 里，能绕开 GIL）；**IO 密集、要频繁共享数据 → 多线程。** ROS2 的节点就是**多进程**的，好处正是**隔离故障**——一个节点挂了别的照跑。

后续待补

- 简历六条项目技术点 + 附录通用追问 + 附录二名词八股 已全部覆盖。
- **网络与并发技能**（epoll/Reactor/生产者-消费者）目前**没有对应项目能支撑**——附录二技术点 4 已补了八股答案，但**背之前先想清楚它来自哪个项目**，别让面试官以为机器人项目里有高并发服务。要么补回一个对应项目，要么把这行降级成"了解"。
- 以后**改了简历措辞**，记得同步对应条目的「简历原话」块和那几道题的说法，别出现简历和口播对不上的情况。
- 项目日期占位 `2026.06 – 2026.10` 记得改成真实时间。

> 补充新题时沿用本文件的格式：**口语问题** → **可直接念的段落** → **追问** → 📌 **自己复习**。 文件名、话题名、参数名一律挪到「📌 自己复习」里，**不进嘴**。 答案控制在 **3~5 句、念完不超过 30 秒**，追问要从上一句答案里的词自然长出来。